

Aula 06 — SQL DDL: do modelo às tabelas no MySQL

Curso: Bacharelado em Sistemas de Informação — 2º período **Disciplina:** Banco de Dados I
Professor: Me. Alysson Martins Bruno **Data:** 14 de setembro de 2026

Como usar este texto

Este material é uma rota de estudo prática: não basta ler, é preciso **digitar os comandos** e comparar o que aparece na sua tela com o que está escrito aqui.

1. Leia uma seção curta.
2. Abra o terminal, suba o MySQL e execute os comandos daquela seção.
3. Confira a saída: em quase todos os comandos, a resposta esperada está reproduzida no texto.
4. Pare nos quadros **Sua vez** e tente responder antes de ler a resposta, que vem logo abaixo.
5. Deu erro? **Leia a mensagem inteira** antes de repetir o comando. A seção 14 reúne os erros mais frequentes com a correção de cada um.
6. Ao terminar, percorra o `laboratorio-bd1-aula06.md`, escreva o script da biblioteca comunitária e responda ao questionário no Google Classroom.

Se você não tem computador em casa, use o laboratório da instituição. Reserve o tempo em blocos: a seção 10 (implementação da clínica) precisa ser feita de uma vez só, do início ao fim.

Tudo o que é necessário para concluir a aula está neste texto, nos slides e no roteiro de laboratório. Dúvida que não se resolver por eles: fórum do Google Classroom, e-mail ou WhatsApp da turma.

1. Introdução

Na Aula 05 percorremos o caminho inteiro da modelagem da clínica veterinária **Patas do Cerrado**: lemos a narrativa do negócio, definimos a fronteira da aplicação, classificamos substantivos e verbos, construímos o Diagrama Entidade-Relacionamento (DER), convertemos o modelo conceitual em modelo lógico e escrevemos, em linguagem natural, a especificação do modelo físico.

Terminamos aquela aula com frases como esta:

“`tutor_id` é obrigatório e deve apontar para um tutor existente.”

Uma frase dessas descreve com precisão uma regra do negócio, mas nenhum computador a executa. Esta aula traduz cada uma dessas frases para a linguagem que o Sistema Gerenciador de Banco de Dados (SGBD) entende:

```
tutor_id BIGINT UNSIGNED NOT NULL,  
CONSTRAINT fk_animal_tutor FOREIGN KEY (tutor_id)  
REFERENCES tutor (id)
```

O conjunto de comandos que cria e altera a **estrutura** do banco chama-se **DDL** (*Data Definition Language*). É o assunto de hoje. Ao final, o modelo desenhado no papel estará no ar, dentro do MySQL, com as regras do negócio protegidas pelo próprio banco.

Um aviso importante sobre o método: **não vamos digitar CREATE TABLE para “ver o que acontece”**. Cada linha do script escrito hoje corresponde a uma decisão tomada na Aula 05. Quando não souber por que uma coluna é obrigatória, volte à especificação — a resposta está lá, não na sintaxe.

2. SQL: origem, padronização e dialetos

2.1 O que é SQL

SQL é a sigla de *Structured Query Language* — Linguagem Estruturada de Consulta. É a linguagem usada para criar, alimentar, consultar e proteger dados em bancos de dados relacionais.

A característica mais importante do SQL é ser uma linguagem **declarativa**: quem escreve descreve **o que** deseja, e não **como** o computador deve fazer. Em Java, você escreve o passo a passo para percorrer uma lista; em SQL, você descreve o resultado desejado e o SGBD decide sozinho o caminho mais eficiente. Essa diferença explica por que um comando SQL é curto e um algoritmo equivalente é longo.

2.2 Uma história em quatro nomes

Quem	Quando	Contribuição
Edgar F. Codd	1970	Propôs o modelo relacional em artigo publicado pela IBM: dados organizados em relações (tabelas), com fundamento matemático
Donald Chamberlin e Raymond Boyce	1974	Criaram o SEQUEL , linguagem para manipular o modelo de Codd; por questão de marca registrada, o nome foi encurtado para SQL
Peter Chen	1976	Publicou o Modelo Entidade-Relacionamento , base do DER usado na Aula 05
Christopher J. Date	anos 1980	Consolidou, sistematizou e divulgou a teoria relacional em obras de referência

Vale perceber a ligação: o DER que você desenhou na Aula 05 e o SQL que vai escrever hoje nasceram da mesma ideia — organizar dados em relações e ligá-las por valores, não por endereços de memória.

2.3 A padronização ANSI/ISO

Nos anos 1980 cada fabricante criou a sua própria versão da linguagem, e um comando escrito para um produto não funcionava em outro. Para resolver esse problema:

- **1986** — o ANSI (*American National Standards Institute*) publica o primeiro padrão, o SQL-86;
- **1987** — a ISO (*International Organization for Standardization*) adota o padrão, tornando-o internacional;
- **revisões seguintes** — SQL-92 (o mais citado como base comum), SQL:1999, SQL:2003, SQL:2008, SQL:2011, SQL:2016 e SQL:2023, acrescentando recursos como gatilhos, tipos estruturados, funções analíticas e suporte a JSON.

2.4 Dialeto

Nenhum SGBD implementa o padrão por inteiro, e todos acrescentam recursos próprios. A essas variações damos o nome de **dialeto**.

Recurso	MySQL	PostgreSQL	Oracle
Numeração automática	AUTO_INCREMENT	SERIAL / IDENTITY	IDENTITY
Texto de tamanho variável	VARCHAR(n)	VARCHAR(n)	VARCHAR2(n)
Limitar o número de linhas	LIMIT 10	LIMIT 10	FETCH FIRST 10 ROWS ONLY
Data e hora do sistema	NOW()	NOW()	SYSDATE

O núcleo é comum: CREATE TABLE, PRIMARY KEY, FOREIGN KEY, NOT NULL e SELECT funcionam em todos. O que muda são as bordas. Por isso o conhecimento desta aula é transferível: mudando de emprego e de banco, você reaproveita o raciocínio e ajusta a escrita.

Nesta disciplina o dialeto é o do **MySQL 8.4**, a mesma versão usada desde a Aula 04.

Sua vez. Um colega afirma: “aprender SQL no MySQL não serve para quem vai trabalhar com PostgreSQL”. Você concorda?

Resposta. Não. O modelo relacional, as restrições de integridade e a estrutura dos comandos são os mesmos; muda a grafia de alguns recursos, como a numeração automática. Quem entende *por que* uma chave estrangeira existe reescreve o comando em qualquer dialeto em poucos minutos.

3. As famílias de comandos SQL

Os comandos SQL são agrupados em quatro famílias, conforme aquilo que manipulam.

Sigla	Nome	O que manipula	Comandos principais
DDL	<i>Data Definition Language</i>	a estrutura do banco	CREATE, ALTER, DROP, TRUNCATE, RENAME
DML	<i>Data Manipulation Language</i>	os dados guardados	INSERT, UPDATE, DELETE, SELECT
DCL	<i>Data Control Language</i>	as permissões de acesso	GRANT, REVOKE
TCL	<i>Transaction Control Language</i>	a confirmação do trabalho	COMMIT, ROLLBACK, SAVEPOINT

Uma analogia ajuda a fixar: a DDL constrói a casa (paredes, cômodos, portas); a DML coloca e retira os móveis; a DCL decide quem recebe a chave de cada porta; a TCL confirma a mudança ou devolve tudo como estava. Nenhum móvel entra antes de existir o cômodo — por isso a DDL vem primeiro, e por isso esta aula precede a Aula 07.

3.1 Um detalhe que assusta quem descobre tarde

No MySQL, os comandos DDL provocam **confirmação automática** (*commit* implícito). Isso significa que:

- um INSERT equivocado pode ser desfeito com ROLLBACK, dentro de uma transação;
- um DROP TABLE **não pode ser desfeito**.

Some-se a isso o fato de que DROP DATABASE apaga tudo sem perguntar duas vezes. A regra prática é simples: **antes de executar qualquer comando que comece com DROP, leia a linha inteira e confirme o nome do objeto**. No laboratório da disciplina, trabalhe apenas nos bancos dos exercícios.

4. Preparando o ambiente

Há dois caminhos para fazer a prática desta aula, e os comandos são os mesmos nos dois.

No navegador. A disciplina tem um ambiente pronto, que abre uma máquina Linux completa dentro do navegador:

<https://labs.iximiuz.com/playgrounds/BD1-AULA06-9e9ee739>

Entre na sua conta do iximiuz Labs e clique em **Start Playground**. A máquina sobe em menos de um minuto, já com a imagem do MySQL 8.4 baixada e com os arquivos da aula em ~/lab06 — compose.yaml, ddl-clinica.sql e conferir-clinica.sql. Nada é instalado no seu computador, o que resolve a vida de quem usa o laboratório da instituição ou uma máquina fraca. Para colar um comando no terminal do navegador, use **Ctrl + Shift + V**; ao encerrar, escolha **Stop**, que guarda o ambiente para você retomar depois, e não **Destroy**, que apaga tudo.

No seu computador. Vale igualmente o ambiente montado na Aula 04. As seções seguintes retomam, em versão curta, o que foi feito lá; se o seu contêiner já existe, pule para a seção 4.3.

4.1 Criar rede e volume (uma única vez)

```
podman network create rede-bd1
podman volume create dados-mysql
```

Rede e volume continuam existindo depois de desligar o computador. O volume é o que guarda os dados: sem ele, tudo o que você criar desaparece quando o contêiner for removido.

4.2 Subir o servidor

```
podman run -d \
  --name mysql-bd1 \
  -e MYSQL_ROOT_PASSWORD=minhasenha \
  -e MYSQL_DATABASE=bd1 \
  -p 3306:3306 \
  -v dados-mysql:/var/lib/mysql \
  --network rede-bd1 \
  mysql:8.4
```

Se preferir, use o arquivo `compose.yaml` que acompanha esta aula — ele guarda exatamente essa configuração:

```
podman compose up -d      # sobe o servidor
podman compose ps         # confere se está no ar
podman compose logs mysql # lê o log de inicialização
podman compose down       # desliga (o volume permanece)
```

Se o contêiner já foi criado em outra aula, não é preciso criá-lo de novo:

```
podman start mysql-bd1
```

4.3 Entrar no cliente

```
podman exec -it mysql-bd1 mysql -u root -p
```

Informe a senha `minhasenha`. Você chega ao prompt do cliente:

```
mysql>
```

Três regras valem para tudo o que vier a seguir:

1. **todo comando termina com ponto e vírgula (;);**
2. se a tela mostrar `->`, o MySQL está esperando o resto do comando — normalmente falta o `;` ou um parêntese;
3. para abandonar um comando começado errado, digite `\c` e Enter; para sair do cliente, digite `exit`.

Sua vez. Você digitou o `CREATE TABLE` inteiro, apertou Enter e a tela apenas mostrou `->`. O comando foi executado?

Resposta. Não. O cliente ainda está lendo o comando, à espera do ponto e vírgula ou do fechamento de um parêntese. Digite `;` e Enter: se o comando estiver completo, ele será executado; se houver parêntese aberto, o `->` volta a aparecer e o caminho é cancelar com `\c` e reescrever.

5. Criando o banco de dados

5.1 CREATE DATABASE

```
CREATE DATABASE IF NOT EXISTS clinica_veterinaria
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_0900_ai_ci;
```

```
Query OK, 1 row affected (0.01 sec)
```

Cada parte do comando:

- **IF NOT EXISTS** — evita erro caso o banco já exista. Útil em scripts executados mais de uma vez.
- **CHARACTER SET utf8mb4** — define o conjunto de caracteres. O `utf8mb4` representa todos os caracteres Unicode, inclusive acentos, ç e emojis. É a escolha recomendada para qualquer projeto novo em português.
- **COLLATE utf8mb4_0900_ai_ci** — define as regras de comparação e de ordenação. O sufixo `ai` significa *accent-insensitive* (ignora acento) e `ci` significa *case-insensitive* (ignora maiúsculas e minúsculas). Com essa configuração, uma busca por `jose` encontra `José`.

Sem `utf8mb4`, nomes com acento podem ser gravados incorretamente ou provocar o erro 1366 (`Incorrect string value`).

5.2 USE e a conferência

Criar o banco não significa estar dentro dele:

```
SHOW DATABASES;
USE clinica_veterinaria;
SELECT DATABASE();
```

A última linha responde:

```
+-----+
| DATABASE() |
+-----+
| clinica_veterinaria |
+-----+
1 row in set (0.00 sec)
```

Adquirar o hábito de rodar `SELECT DATABASE();` antes de criar tabelas. Esquecer o USE produz o erro:

```
ERROR 1046 (3D000): No database selected
```

E, pior do que o erro, é o caso em que o USE aponta para o banco errado: as tabelas são criadas em outro lugar, sem aviso nenhum.

6. Tipos de dados no MySQL

6.1 Por que o tipo importa

O tipo de dado de uma coluna responde a três perguntas ao mesmo tempo:

1. **o que cabe ali** — só números? só datas? qualquer texto?
2. **quanto espaço** cada linha vai ocupar em disco;
3. **o que o banco deve recusar** na hora de gravar.

O tipo é, portanto, a **primeira restrição** de uma coluna. Declarar `data_nascimento DATE` já impede que alguém grave “ontem de tarde” nesse campo.

6.2 Tipos numéricos inteiros

Tipo	Bytes	Intervalo com sinal	Uso recomendado
TINYINT	1	−128 a 127	indicadores, idade
SMALLINT	2	−32.768 a 32.767	quantidades pequenas
MEDIUMINT	3	±8,3 milhões	contadores médios
INT	4	±2,1 bilhões	códigos, quantidades
BIGINT	8	±9,2 × 10 ¹⁸	chaves primárias de tabelas grandes

O qualificador **UNSIGNED** elimina os valores negativos e dobra o limite positivo: `SMALLINT UNSIGNED` vai de 0 a 65.535. Usamos `BIGINT UNSIGNED` nas chaves primárias da clínica porque identificador não é negativo e porque o custo de mudar o tipo depois, com a tabela cheia, é alto.

6.3 Tipos numéricos com casas decimais

Tipo	Como armazena	Use para
DECIMAL (p, s)	exato: <i>p</i> dígitos no total, <i>s</i> deles depois da vírgula	dinheiro, notas, percentuais
FLOAT	aproximado, 4 bytes	medidas científicas
DOUBLE	aproximado, 8 bytes	medidas científicas de maior precisão

FLOAT e DOUBLE guardam números em ponto flutuante binário e, por isso, têm imprecisão inerente: a soma $0,1 + 0,2$ não resulta exatamente em $0,3$. Em um sistema com milhares de lançamentos, essas frações se acumulam e o caixa fecha com diferença.

Regra sem exceção nesta disciplina: dinheiro em DECIMAL. Na clínica, DECIMAL (10,2) comporta até 99.999.999,99 — mais do que suficiente.

6.4 Tipos de texto

Tipo	Limite	Característica	Uso recomendado
CHAR(n)	255 caracteres	tamanho fixo , completado com espaços	UF, CPF, CEP, siglas
VARCHAR(n)	65.535 bytes por linha	tamanho variável	nome, e-mail, endereço
TEXT	65.535 caracteres	texto longo, sem tamanho declarado	observações, descrições
MEDIUMTEXT / LONGTEXT	16 MB / 4 GB	textos muito longos	documentos inteiros

CHAR(n) ocupa sempre *n* posições, mesmo que o valor seja menor; VARCHAR(n) ocupa o que o valor exigir, mais um ou dois bytes de controle. Para conteúdo de tamanho realmente fixo, como a UF (2 letras) ou o CPF (11 dígitos), CHAR é ligeiramente mais eficiente. Para nomes, VARCHAR é a escolha natural.

Atenção a um engano comum: **CPF, CEP, telefone e CRMV são texto, não número.** Eles têm zeros à esquerda, aceitam pontuação e nunca entram em conta de somar.

6.5 Tipos de data e hora

Tipo	Formato	Intervalo	Uso recomendado
DATE	AAAA-MM-DD	1000-01-01 a 9999-12-31	data de nascimento, vencimento
TIME	HH:MM:SS	-838:59:59 a 838:59:59	duração, horário isolado
DATETIME	AAAA-MM-DD HH:MM:SS	1000 a 9999	momento em que um evento ocorreu
TIMESTAMP	idem, convertido por fuso	1970 a 2038	auditoria: criado em, alterado em
YEAR	AAAA	1901 a 2155	ano de publicação

Duas observações práticas:

- **no SQL, a data se escreve sempre no formato '2026-09-14'** — ano, mês e dia, nessa ordem, entre aspas simples. A escrita '14/09/2026' provoca o erro 1292 (Incorrect date value);
- DATETIME guarda o valor literal; TIMESTAMP guarda internamente em UTC e converte conforme o fuso do servidor. Para o horário de uma consulta veterinária, DATETIME é o adequado.

6.6 Outros tipos úteis

- **BOOLEAN** — no MySQL é apelido de TINYINT(1): 0 é falso, qualquer outro valor é verdadeiro.
- **ENUM('a', 'b', 'c')** — aceita apenas um valor de uma lista fixa. Serve para situações estáveis, como ENUM('agendada', 'realizada', 'cancelada'). Mudar a lista depois exige ALTER TABLE, o que torna o ENUM inadequado quando as opções variam com frequência.
- **BLOB** — dados binários. Em geral é preferível guardar o arquivo no sistema de arquivos e apenas o caminho no banco.

6.7 Como escolher, em três perguntas

1. O conteúdo é **número de contar ou calcular** ou é **código**? Código é texto.
2. Tem **casas decimais e representa dinheiro**? Então DECIMAL.
3. O tamanho é **sempre o mesmo**? Então CHAR; caso contrário, VARCHAR.

Na dúvida entre VARCHAR(50) e VARCHAR(100), escolha o que comporta o **maior caso real**, não o caso médio: o espaço só é ocupado quando o valor existe.

Sua vez. Escolha o tipo de cada coluna e justifique: (a) crmv do veterinário; (b) valor_praticado do procedimento; (c) data_hora da consulta; (d) observacoes da consulta.

Resposta. (a) VARCHAR(20): é código, com letras e separadores, e não se calcula. (b) DECIMAL(10,2): é dinheiro e precisa ser exato. (c) DATETIME: o negócio precisa da data e da hora do atendimento. (d) TEXT: texto livre, de tamanho imprevisível.

7. O comando CREATE TABLE

7.1 Forma geral

```
CREATE TABLE nome_da_tabela (  
    coluna1 TIPO [restrições da coluna],  
    coluna2 TIPO [restrições da coluna],  
    CONSTRAINT nome_da_restricao TIPO_DA_RESTRICAO (colunas)  
);
```

Regras de escrita que evitam a maior parte dos erros de sintaxe:

- cada coluna em uma linha, separada da seguinte por **vírgula**;
- **não** há vírgula antes do parêntese que fecha a lista;
- as restrições podem ser declaradas junto da coluna (NOT NULL, DEFAULT) ou ao final, com nome próprio (CONSTRAINT ...);
- o comando termina com);.

7.2 Convenções de nomenclatura da disciplina

Elemento	Convenção	Exemplo
Tabela	substantivo no singular, minúsculas	animal, consulta_procedimento
Coluna	minúsculas, palavras separadas por sublinhado	data_nascimento
Chave primária	pk_ + tabela	pk_tutor
Chave estrangeira	fk_ + tabela filha + tabela pai	fk_animal_tutor
Unicidade	uq_ + tabela + coluna	uq_tutor_email
Verificação	ck_ + tabela + regra	ck_cp_quantidade

Nomes de tabela e de coluna vão **sem acento e sem espaço**. O conteúdo guardado tem acento; o nome da estrutura, não — é o que evita problemas ao trocar de sistema operacional, de SGBD ou de linguagem de programação.

7.3 NULL e NOT NULL

NULL não é zero nem texto vazio: significa “**valor não informado**”.

Situação	O que guardar
tutor que não forneceu e-mail	NULL
procedimento gratuito	0.00
observação deixada em branco de propósito	' '

Declarar NOT NULL torna a coluna **obrigatória**: a linha não é gravada sem esse valor. Omitir o NOT NULL torna a coluna opcional.

Cada NOT NULL é uma decisão de negócio. Na clínica, nome do animal é obrigatório (sem ele, o prontuário não identifica o paciente), mas raça é opcional (um animal resgatado pode chegar sem essa informação).

7.4 DEFAULT

```
quantidade SMALLINT UNSIGNED NOT NULL DEFAULT 1,
```

O DEFAULT define o valor usado quando o comando de inserção não informa a coluna. Na tabela associativa da clínica, o padrão 1 traduz uma expectativa do negócio: quando o procedimento é lançado, ele foi feito uma vez, salvo indicação em contrário.

7.5 AUTO_INCREMENT

```
id BIGINT UNSIGNED AUTO_INCREMENT,
```

O `AUTO_INCREMENT` faz o MySQL gerar os valores 1, 2, 3... automaticamente. É assim que o `id` artificial adotado na Aula 05 se preenche sem trabalho. Duas exigências:

- a coluna precisa ser **chave** — na prática, a chave primária;
- só pode haver **uma** coluna `AUTO_INCREMENT` por tabela.

Um número consumido não volta: se uma inserção falhar, aquele valor é descartado e o próximo `id` pula. Isso é normal e não indica erro — identificador serve para identificar, não para contar.

8. Restrições: as regras do negócio dentro do banco

Restrição (*constraint*) é a regra que o banco aplica a **toda** gravação, venha ela de um sistema, de um relatório ou de alguém digitando no terminal. Enquanto uma validação escrita no programa protege apenas aquele programa, a restrição protege o dado em qualquer caminho de entrada.

Cada restrição responde a uma pergunta:

Restrição	Pergunta do negócio
NOT NULL	esse dado pode faltar?
PRIMARY KEY	o que identifica cada linha?
UNIQUE	esse valor pode se repetir?
CHECK	que valores fazem sentido?
FOREIGN KEY	essa linha depende de qual outra?

8.1 PRIMARY KEY

```
CONSTRAINT pk_tutor PRIMARY KEY (id)
```

A chave primária identifica cada linha de forma única. Ela **nunca aceita NULL**, mesmo que o `NOT NULL` não seja escrito, e só pode haver uma por tabela — ainda que possa envolver mais de uma coluna.

Um detalhe do MySQL que costuma confundir: o nome dado à chave primária **é ignorado**. Internamente ela sempre se chama `PRIMARY`. Continuamos escrevendo `CONSTRAINT pk_tutor` por dois motivos: documenta a intenção para quem lê o script e mantém o padrão de escrita das demais restrições, cujos nomes o banco de fato guarda.

8.2 UNIQUE

```
CONSTRAINT uq_veterinario_crmv UNIQUE (crmv)
```

Impede repetição de valor na coluna. Diferenças em relação à chave primária: pode haver **várias** restrições UNIQUE na mesma tabela e ela **aceita NULL** — no MySQL, inclusive vários NULL, porque valores não informados não são considerados iguais entre si.

A unicidade também pode combinar colunas:

```
CONSTRAINT uq_consulta_agenda UNIQUE (veterinario_id, data_hora)
```

Essa linha diz, em SQL, que **o mesmo veterinário não atende duas consultas no mesmo horário** — mas nada impede que dois veterinários diferentes atendam às 14h.

8.3 Chave artificial e chave natural convivem

Pergunta que sempre aparece: “*se a tabela já tem id, o CRMV ainda precisa de restrição?*”

Precisa. São coisas diferentes:

- o **id** é a chave **técnica**: curta, estável, usada para ligar as tabelas;
- o **CRMV** é a chave **natural**: a regra do mundo real, que não admite dois profissionais com o mesmo registro.

Sem o UNIQUE (crmv), o banco aceitaria dois veterinários com o mesmo CRMV e id diferentes — dois cadastros para a mesma pessoa. O mesmo raciocínio vale para o ISBN de um livro e para o e-mail do tutor.

8.4 CHECK

```
CONSTRAINT ck_procedimento_valor CHECK (valor_referencia >= 0),  
CONSTRAINT ck_cp_quantidade CHECK (quantidade > 0)
```

O CHECK impõe uma condição sobre o valor gravado, verificada em toda inserção e em toda alteração. No MySQL, a restrição passou a ser **efetivamente aplicada** a partir da versão 8.0.16; em versões anteriores ela era aceita na sintaxe e ignorada na prática — motivo pelo qual ainda circulam materiais afirmando que “CHECK não funciona no MySQL”.

A tentativa de gravar valor negativo produz:

```
ERROR 3819 (HY000): Check constraint 'ck_procedimento_valor' is violated.
```

8.5 FOREIGN KEY

```
tutor_id BIGINT UNSIGNED NOT NULL,  
CONSTRAINT fk_animal_tutor FOREIGN KEY (tutor_id)  
REFERENCES tutor (id)
```

A chave estrangeira estabelece o vínculo entre duas tabelas e garante a **integridade referencial**: o valor guardado em `animal.tutor_id` precisa existir em `tutor.id`. É o que impede o banco de ter um animal sem tutor — o famoso registro órfão.

O MySQL exige três condições:

1. a tabela referenciada **já precisa existir** quando o comando é executado;
2. os tipos das duas colunas devem ser **iguais**, inclusive quanto ao UNSIGNED;
3. a coluna referenciada precisa ser **chave primária ou UNIQUE**.

A combinação NOT NULL + FOREIGN KEY traduz a participação obrigatória definida no DER: *todo animal tem um tutor, e esse tutor existe*.

8.6 ON DELETE e ON UPDATE

```
CONSTRAINT fk_animal_tutor FOREIGN KEY (tutor_id)
REFERENCES tutor (id)
ON DELETE RESTRICT
ON UPDATE CASCADE
```

Essas cláusulas definem o que acontece com a linha filha quando a linha pai é apagada ou tem sua chave alterada:

Ação	Efeito
RESTRICT	impede a operação enquanto existirem filhos (é o padrão)
CASCADE	repete a operação nos filhos: apaga junto ou atualiza junto
SET NULL	grava NULL na coluna do filho — exige coluna opcional
NO ACTION	no MySQL, comporta-se como RESTRICT

Padrão adotado na disciplina: ON DELETE RESTRICT e ON UPDATE CASCADE. O RESTRICT protege o histórico: ninguém apaga um tutor e leva junto o prontuário dos animais dele. O CASCADE no UPDATE é inofensivo aqui, porque a chave é artificial e praticamente nunca muda.

Há uma exceção proposital no modelo da clínica: em `consulta_procedimento`, a chave estrangeira para `consulta` usa ON DELETE CASCADE. A razão é que o item de procedimento **não existe sem a consulta** — apagada a consulta, os itens dela perdem o sentido. Já a chave estrangeira para `procedimento` continua RESTRICT, porque o catálogo de procedimentos é independente e não pode ser afetado por uma consulta apagada.

Sua vez. Um colega escreveu ON DELETE CASCADE na chave estrangeira de `consulta` para `animal`. Que consequência isso traz?

Resposta. Apagar um animal apagaria, em silêncio, todo o histórico de consultas dele — e, por efeito em cadeia, os procedimentos dessas consultas. Em prontuário clínico isso é inaceitável: o correto é

RESTRICT, que impede apagar o animal enquanto houver consultas registradas.

9. A ordem de criação das tabelas

Como a chave estrangeira exige que a tabela referenciada já exista, a ordem dos comandos deixa de ser detalhe e passa a ser regra:

crie primeiro as tabelas que não dependem de ninguém.

Para a clínica, a sequência é:

1. tutor, veterinario e procedimento — nenhuma chave estrangeira;
2. animal — depende de tutor;
3. consulta — depende de animal e de veterinario;
4. consulta_procedimento — depende de consulta e de procedimento.

Para **apagar**, o caminho é o inverso: primeiro as tabelas que dependem das outras. Tentar apagar tutor antes de animal produz erro, e é bom que produza — é a integridade referencial funcionando.

Uma forma prática de descobrir a ordem em qualquer modelo: desenhe uma seta de cada tabela para aquelas que ela referencia. As tabelas de onde não sai nenhuma seta são as primeiras a criar.

10. Implementando a clínica Patas do Cerrado

Esta seção é a parte central da aula. Execute os comandos **na ordem em que aparecem**, conferindo a resposta de cada um. O script completo também está no arquivo `ddl-clinica.sql`, mas digite pelo menos as três primeiras tabelas: a memória do comando se forma na digitação.

10.1 Passo 1 — o banco

```
CREATE DATABASE IF NOT EXISTS clinica_veterinaria
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_0900_ai_ci;

USE clinica_veterinaria;
SELECT DATABASE();
```

Confirme que a resposta é `clinica_veterinaria` antes de seguir.

10.2 Passo 2 — tutor

Especificação da Aula 05: *o tutor tem nome e telefone obrigatórios; o e-mail é opcional, mas, quando informado, não pode se repetir.*

```
CREATE TABLE tutor (
  id          BIGINT UNSIGNED AUTO_INCREMENT,
  nome        VARCHAR(100)    NOT NULL,
  telefone    VARCHAR(20)     NOT NULL,
  email       VARCHAR(150),
  CONSTRAINT pk_tutor        PRIMARY KEY (id),
  CONSTRAINT uq_tutor_email  UNIQUE (email)
);
```

Query OK, 0 rows affected (0.04 sec)

A resposta 0 rows affected está correta: a DDL cria estrutura, não linhas.

Linha a linha: id é a chave técnica, numerada automaticamente; nome e telefone são obrigatórios porque a clínica precisa localizar o responsável pelo animal; email não tem NOT NULL, e por isso aceita NULL; o UNIQUE impede dois cadastros com o mesmo e-mail — e, como o MySQL admite vários NULL em coluna UNIQUE, continua sendo possível cadastrar muitos tutores sem e-mail.

10.3 Passo 3 — veterinario

Especificação: *nome e CRMV obrigatórios; o CRMV identifica o profissional e não se repete.*

```
CREATE TABLE veterinario (
  id          BIGINT UNSIGNED AUTO_INCREMENT,
  nome        VARCHAR(100)    NOT NULL,
  crmv        VARCHAR(20)     NOT NULL,
  especialidade VARCHAR(80),
  CONSTRAINT pk_veterinario    PRIMARY KEY (id),
  CONSTRAINT uq_veterinario_crmv UNIQUE (crmv)
);
```

especialidade ficou opcional: nem todo veterinário tem especialização registrada.

10.4 Passo 4 — procedimento

Especificação: *cada procedimento tem um nome que não se repete e um valor de referência que nunca é negativo.*

```
CREATE TABLE procedimento (
  id          BIGINT UNSIGNED AUTO_INCREMENT,
  nome        VARCHAR(100)    NOT NULL,
  descricao   VARCHAR(255),
  valor_referencia DECIMAL(10,2) NOT NULL,
  CONSTRAINT pk_procedimento    PRIMARY KEY (id),
  CONSTRAINT uq_procedimento_nome UNIQUE (nome),
  CONSTRAINT ck_procedimento_valor CHECK (valor_referencia >= 0)
);
```

Três regras, três restrições: identidade (PRIMARY KEY), nome que não repete (UNIQUE) e valor que não fica negativo (CHECK). Repare que a regra escolhida foi ≥ 0 , e não > 0 : a clínica pode registrar um procedimento de cortesia com valor zero.

10.5 Passo 5 — animal

Especificação: *todo animal pertence a um tutor existente; nome e espécie são obrigatórios; raça e data de nascimento podem faltar.*

```
CREATE TABLE animal (  
  id                BIGINT UNSIGNED AUTO_INCREMENT,  
  tutor_id          BIGINT UNSIGNED NOT NULL,  
  nome              VARCHAR(80)    NOT NULL,  
  especie           VARCHAR(40)    NOT NULL,  
  raca              VARCHAR(60),  
  data_nascimento   DATE,  
  CONSTRAINT pk_animal PRIMARY KEY (id),  
  CONSTRAINT fk_animal_tutor FOREIGN KEY (tutor_id)  
    REFERENCES tutor (id)  
    ON DELETE RESTRICT  
    ON UPDATE CASCADE  
);
```

Aqui aparece a primeira chave estrangeira. Ela é a tradução direta do relacionamento **TUTOR 1:N ANIMAL** do DER: a chave migrou para o lado N, exatamente como decidido na Aula 05. O NOT NULL em tutor_id representa a participação obrigatória — não existe animal sem tutor.

10.6 Passo 6 — consulta

Especificação: *toda consulta pertence a um animal e a um veterinário; data/hora e motivo são obrigatórios; o mesmo veterinário não pode ter duas consultas no mesmo horário.*

```

CREATE TABLE consulta (
  id          BIGINT UNSIGNED AUTO_INCREMENT,
  animal_id   BIGINT UNSIGNED NOT NULL,
  veterinario_id BIGINT UNSIGNED NOT NULL,
  data_hora   DATETIME        NOT NULL,
  motivo      VARCHAR(255)    NOT NULL,
  observacoes TEXT,
  CONSTRAINT pk_consulta PRIMARY KEY (id),
  CONSTRAINT fk_consulta_animal FOREIGN KEY (animal_id)
    REFERENCES animal (id)
    ON DELETE RESTRICT
    ON UPDATE CASCADE,
  CONSTRAINT fk_consulta_veterinario FOREIGN KEY (veterinario_id)
    REFERENCES veterinario (id)
    ON DELETE RESTRICT
    ON UPDATE CASCADE,
  CONSTRAINT uq_consulta_agenda UNIQUE (veterinario_id, data_hora)
);

```

Duas chaves estrangeiras, porque a consulta está no lado N de dois relacionamentos 1:N. E uma restrição de unicidade composta, que protege a agenda: o par veterinário + horário é único.

10.7 Passo 7 — consulta_procedimento

Especificação: *um procedimento aparece uma única vez em cada consulta; a repetição é registrada na quantidade; a quantidade é maior que zero; o valor praticado nunca é negativo.*

```

CREATE TABLE consulta_procedimento (
  id          BIGINT UNSIGNED AUTO_INCREMENT,
  consulta_id  BIGINT UNSIGNED NOT NULL,
  procedimento_id BIGINT UNSIGNED NOT NULL,
  quantidade   SMALLINT UNSIGNED NOT NULL DEFAULT 1,
  valor_praticado DECIMAL(10,2) NOT NULL,
  CONSTRAINT pk_consulta_procedimento PRIMARY KEY (id),
  CONSTRAINT fk_cp_consulta FOREIGN KEY (consulta_id)
    REFERENCES consulta (id)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
  CONSTRAINT fk_cp_procedimento FOREIGN KEY (procedimento_id)
    REFERENCES procedimento (id)
    ON DELETE RESTRICT
    ON UPDATE CASCADE,
  CONSTRAINT uq_consulta_procedimento UNIQUE (consulta_id, procedimento_id),
  CONSTRAINT ck_cp_quantidade CHECK (quantidade > 0),
  CONSTRAINT ck_cp_valor CHECK (valor_praticado >= 0)
);

```

Esta é a tabela associativa que resolveu o relacionamento **N:M** entre consulta e procedimento. Três decisões merecem atenção:

- **UNIQUE (consulta_id, procedimento_id)** — o mesmo procedimento não entra duas vezes na mesma consulta; se foi feito duas vezes, quantidade = 2;
- **ON DELETE CASCADE para a consulta** — o item não existe sem a consulta;
- **valor_praticado separado de valor_referencia** — o catálogo guarda o preço atual, a linha da consulta guarda o que foi efetivamente cobrado naquele atendimento. Preço de tabela muda; fato histórico não. Esse cuidado aparece em qualquer sistema com preço: venda, contrato, matrícula.

Sua vez. Por que não colocamos CHECK (valor_praticado = valor_referencia)?

Resposta. Porque o valor praticado pode legitimamente diferir do de referência: desconto, campanha de vacinação, acordo com protetor de animais. A restrição transformaria uma prática comum da clínica em erro do sistema. Restrição representa regra do negócio — e a regra aqui é apenas que o valor não seja negativo.

11. Conferindo o resultado

11.1 As tabelas criadas

```
SHOW TABLES;
```

```
+-----+
| Tables_in_clinica_veterinaria |
+-----+
| animal                        |
| consulta                     |
| consulta_procedimento        |
| procedimento                 |
| tutor                        |
| veterinario                  |
+-----+
6 rows in set (0.00 sec)
```

Seis tabelas, em ordem alfabética. Se faltar alguma, volte ao passo correspondente e leia a mensagem de erro que apareceu lá.

11.2 A estrutura de uma tabela

```
DESCRIBE animal;
```

Field	Type	Null	Key	Default	Extra
id	bigint unsigned	NO	PRI	NULL	auto_increment
tutor_id	bigint unsigned	NO	MUL	NULL	
nome	varchar(80)	NO		NULL	
especie	varchar(40)	NO		NULL	
raca	varchar(60)	YES		NULL	
data_nascimento	date	YES		NULL	

6 rows in set (0.00 sec)

Como ler essa saída:

- **Null** — NO significa coluna obrigatória; YES, opcional. Compare com a especificação: `raca` e `data_nascimento` são as duas únicas com YES;
- **Key** — PRI é chave primária, UNI é restrição de unicidade, MUL indica a primeira coluna de um índice que aceita repetição, caso típico da chave estrangeira;
- **Extra** — mostra o `auto_increment`.

11.3 A tabela como o banco a guardou

```
SHOW CREATE TABLE animal\G
```

Esse comando devolve o `CREATE TABLE` completo, já com tudo o que o MySQL acrescentou: o motor de armazenamento (InnoDB), o conjunto de caracteres e as restrições com seus nomes. É a forma mais confiável de conferir se uma chave estrangeira ficou como você pretendia.

O `\G` no lugar do `;` troca a saída em tabela por uma lista vertical, bem mais legível em telas estreitas — inclusive no celular.

11.4 Todas as restrições de uma vez

```
SELECT table_name, constraint_name, constraint_type
FROM information_schema.table_constraints
WHERE table_schema = 'clinica_veterinaria'
ORDER BY table_name, constraint_type;
```

No ambiente do navegador, essa e outras conferências saem prontas em um único comando, executado dentro do cliente MySQL:

```
SOURCE /home/laborant/lab06/conferir-clinica.sql;
```

O script não altera nada: apenas lê a estrutura e mostra as seis tabelas esperadas (marcando `>>> FALTA` no que não foi criado), as colunas com a obrigatoriedade de cada uma, as restrições, as chaves estrangeiras com as regras `ON DELETE` e `ON UPDATE` e os `CHECK`.

O `information_schema` é um banco interno do MySQL que guarda **metadados**: dados sobre os dados. É de onde vêm as informações que o `DESCRIBE` resume. A consulta acima lista, de uma vez, todas as restrições do banco — chaves primárias, chaves estrangeiras, unicidades e verificações —, o que é útil para conferir o próprio trabalho antes de entregar. As chaves primárias aparecem sempre com o nome `PRIMARY`, pelo motivo explicado na seção 8.1.

12. Testando as restrições de propósito

A melhor forma de acreditar que uma restrição funciona é tentar quebrá-la. Os comandos desta seção usam `INSERT`, que é DML e será estudado na Aula 07; aqui eles servem apenas como instrumento de teste — copie e execute sem se preocupar ainda com a sintaxe.

Teste 1 — chave estrangeira inexistente

```
INSERT INTO animal (tutor_id, nome, especie) VALUES (999, 'Rex', 'Canino');
```

```
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails
(`clinica_veterinaria`.`animal`, CONSTRAINT `fk_animal_tutor` FOREIGN KEY
(`tutor_id`)
REFERENCES `tutor` (`id`) ON DELETE RESTRICT ON UPDATE CASCADE)
```

Não existe tutor com id 999, e o banco recusou o registro órfão. A mensagem ainda informa qual restrição foi violada.

Teste 2 — coluna obrigatória

```
INSERT INTO tutor (telefone) VALUES ('639999990000');
```

```
ERROR 1364 (HY000): Field 'nome' doesn't have a default value
```

Teste 3 — valor fora da regra

```
INSERT INTO procedimento (nome, valor_referencia) VALUES ('Consulta', -50);
```

```
ERROR 3819 (HY000): Check constraint 'ck_procedimento_valor' is violated.
```

Teste 4 — repetição indevida

Insira o mesmo veterinário duas vezes com o mesmo CRMV:

```
INSERT INTO veterinario (nome, crmv) VALUES ('Ana Souza', 'T0-1234');
INSERT INTO veterinario (nome, crmv) VALUES ('Ana S. Souza', 'T0-1234');
```

```
ERROR 1062 (23000): Duplicate entry 'T0-1234' for key  
'veterinario.uq_veterinario_crmv'
```

A primeira inserção funciona; a segunda é recusada. Quatro erros, quatro regras do negócio protegidas. **Essa é a diferença entre um banco de dados e uma planilha.**

Para deixar o banco limpo antes do laboratório, remova o veterinário de teste:

```
DELETE FROM veterinario WHERE crmv = 'T0-1234';
```

13. Corrigindo sem recomeçar

Descobrir um erro na estrutura é rotina de projeto. Não é preciso apagar o banco e começar de novo.

13.1 ALTER TABLE

```
-- acrescentar uma coluna  
ALTER TABLE tutor ADD COLUMN cidade VARCHAR(60);  
  
-- mudar tipo ou obrigatoriedade de uma coluna existente  
ALTER TABLE animal MODIFY COLUMN raca VARCHAR(80) NOT NULL;  
  
-- renomear uma coluna  
ALTER TABLE tutor RENAME COLUMN telefone TO celular;  
  
-- acrescentar uma restrição  
ALTER TABLE tutor ADD CONSTRAINT uq_tutor_celular UNIQUE (celular);  
  
-- remover uma restrição  
ALTER TABLE tutor DROP CONSTRAINT uq_tutor_celular;  
  
-- remover uma coluna (apaga também os dados dela)  
ALTER TABLE tutor DROP COLUMN cidade;
```

Os comandos acima são exemplos de sintaxe, e não um roteiro a executar em sequência. Se você os testar no banco da clínica, desfça as alterações ao final — `ALTER TABLE tutor RENAME COLUMN celular TO telefone;` — para que a estrutura continue igual à do modelo da Aula 05.

Uma advertência: `ALTER TABLE` em tabela com dados pode **falhar** se os dados existentes contrariarem a nova regra. Tornar `raca` obrigatória quando já existem animais sem raça cadastrada produz erro — e é justamente isso que se espera de um banco íntegro.

13.2 DROP TABLE e TRUNCATE TABLE

```
DROP TABLE IF EXISTS consulta_procedimento;  -- apaga estrutura e dados
TRUNCATE TABLE consulta_procedimento;        -- apaga só os dados
```

- DROP TABLE desmonta a tabela inteira e **não pode ser desfeito**;
- TRUNCATE TABLE esvazia a tabela, mantém a estrutura e reinicia a contagem do AUTO_INCREMENT;
- tabela referenciada por chave estrangeira não é apagada:

```
ERROR 3730 (HY000): Cannot drop table 'tutor' referenced by a foreign key
constraint
'fk_animal_tutor' on table 'animal'.
```

Sua vez. Você quer recomeçar o exercício apagando as seis tabelas. Em que ordem?

Resposta. Na ordem inversa da criação: consulta_procedimento, depois consulta, depois animal e, por fim, tutor, veterinario e procedimento. Começando por tutor, o MySQL recusa com o erro 3730 — o que é um bom sinal, não um problema.

14. Erros mais frequentes

Mensagem	O que significa	Como corrigir
1046 (3D000): No database selected	faltou escolher o banco	execute <code>USE clinica_veterinaria;</code>
1049 (42000): Unknown database	o banco não existe ou o nome está errado	confira com <code>SHOW DATABASES;</code>
1064 (42000): You have an error in your SQL syntax	erro de digitação	vírgula sobrando antes do <code>)</code> , parêntese aberto ou palavra errada; olhe o trecho citado e a linha anterior
1050 (42S01): Table already exists	a tabela já foi criada	confira com <code>SHOW TABLES;</code>
1824 (HY000): Failed to open the referenced table	a tabela apontada pela FK ainda não existe	crie a tabela pai primeiro
3780 (HY000): ... are incompatible	tipos diferentes entre FK e coluna referenciada	iguale os tipos, inclusive o <code>UNSIGNED</code>
1452 (23000): a foreign key constraint fails	o valor da FK não existe na tabela pai	insira antes a linha pai, ou corrija o valor
1062 (23000): Duplicate entry	violação de <code>PRIMARY KEY</code> ou <code>UNIQUE</code>	o valor já existe; verifique se não é cadastro repetido
3819 (HY000): Check constraint is violated	o valor contraria um <code>CHECK</code>	corrija o valor ou reveja a regra
1364 (HY000): Field doesn't have a default value	coluna <code>NOT NULL</code> não informada	informe a coluna na inserção
1366 (HY000): Incorrect string value	caractere fora do conjunto de caracteres	crie o banco com <code>utf8mb4</code>
1292 (22007): Incorrect date value	data em formato inválido	escreva <code>'AAAA-MM-DD'</code>
3730 (HY000): Cannot drop table ... referenced	há FK apontando para a tabela	apague antes as tabelas filhas

Como ler qualquer mensagem do MySQL

ERROR 1824 (HY000): Failed to open the referenced table 'tutor'

____/	_____	____/
número	o que houve	objeto

1. o **número** identifica o tipo de problema e permite pesquisar na documentação;
2. o **texto** diz o que o servidor não conseguiu fazer;
3. o **objeto citado** mostra onde olhar primeiro.

Erro não é castigo: é o banco recusando um dado que violaria uma regra. Leia, corrija, execute de novo.

15. Atividade guiada: a biblioteca comunitária

Ao final da Aula 05 você modelou uma biblioteca comunitária e chegou a este modelo lógico:

```
LEITOR (id PK, nome, telefone)
LIVRO (id PK, titulo, isbn UQ)
EXEMPLAR (id PK, livro_id FK, codigo_tombo UQ)
EMPRESTIMO (id PK, leitor_id FK, exemplar_id FK,
            data_retirada, data_devolucao_prevista,
            data_devolucao_real)
```

Tarefa: escrever o script DDL completo desse modelo, em um banco chamado biblioteca.

Etapa A — a ordem

Antes de digitar, responda: qual tabela pode ser criada primeiro? Desenhe as setas de dependência.

Resposta: leitor e livro não dependem de ninguém; exemplar depende de livro; emprestimo depende de leitor e de exemplar.

Etapa B — os tipos

Coluna	Tipo sugerido	Por quê
id (todas)	BIGINT UNSIGNED	identificador, nunca negativo
nome, titulo	VARCHAR(120)	texto de tamanho variável
telefone	VARCHAR(20)	código, não número
isbn	VARCHAR(20)	código com hífen
codigo_tombo	VARCHAR(20)	código interno da biblioteca
datas	DATE	dia, sem hora

Etapa C — as restrições

- PRIMARY KEY em todos os id;
- NOT NULL em nome, titulo, codigo_tombo, data_retirada e data_devolucao_prevista;
- UNIQUE em isbn e em codigo_tombo;
- FOREIGN KEY em livro_id, leitor_id e exemplar_id;
- data_devolucao_real **opcional**.

Etapa D — duas decisões para justificar

1. **Por que data_devolucao_real aceita NULL?** Porque NULL aqui significa “*ainda não devolveu*”. É exatamente o que distingue um empréstimo em aberto de um encerrado — e é assim que, na Aula 07, a biblioteca vai conseguir listar os atrasos.
2. **Que ON DELETE usar em emprestimo?** RESTRICT nas duas chaves estrangeiras: o histórico de empréstimos não pode desaparecer porque alguém apagou o cadastro de um leitor ou removeu um

exemplar do acervo.

Etapa E — conferência

Depois de executar o seu script, confira com:

```
USE biblioteca;
SHOW TABLES;          -- quatro tabelas
DESCRIBE emprestimo;    -- três FKs? datas certas? só data_devolucao_real com Null
= YES?
```

O script completo, comentado, é publicado no Google Classroom após o prazo da atividade. Compare o seu com ele — mas só depois de tentar.

16. Checklist do script DDL

Use esta lista sempre que escrever um DDL, aqui e no trabalho da A2:

- ☐ o banco foi criado com `CHARACTER SET utf8mb4`;
 - ☐ o `USE` foi executado e `SELECT DATABASE()` confirma o banco certo;
 - ☐ toda tabela tem `id` e `PRIMARY KEY`;
 - ☐ as tabelas foram criadas na ordem das dependências;
 - ☐ cada chave estrangeira aponta para uma coluna `PK` ou `UNIQUE`, do **mesmo tipo**;
 - ☐ cada coluna obrigatória no modelo tem `NOT NULL`;
 - ☐ cada chave natural do negócio tem `UNIQUE`;
 - ☐ cada regra de valor virou `CHECK`;
 - ☐ valores monetários usam `DECIMAL`, nunca `FLOAT`;
 - ☐ códigos (CPF, CRMV, ISBN, telefone) são texto;
 - ☐ as restrições foram nomeadas no padrão `pk_`, `fk_`, `uq_`, `ck_`;
 - ☐ `SHOW TABLES` mostra o número esperado de tabelas;
 - ☐ `DESCRIBE` de cada tabela confere com o modelo lógico.
-

17. Síntese

O SQL nasceu do modelo relacional proposto por Codd e foi padronizado pela ANSI e pela ISO, o que garante um núcleo comum a todos os bancos relacionais, apesar dos dialetos de cada fabricante. Seus comandos se agrupam em quatro famílias, e a DDL é a que define a estrutura.

Criar tabelas é traduzir um modelo, não improvisar sintaxe: cada tipo de dado delimita o que a coluna aceita, e cada restrição registra dentro do banco uma regra do negócio — obrigatoriedade, identidade, unicidade, valores válidos e dependência entre tabelas. Por causa das chaves estrangeiras, a ordem de criação segue as dependências, e a ordem de exclusão é a inversa.

O banco recusa o que viola as regras, e faz isso por meio de mensagens que dizem o número do erro, o que aconteceu e qual objeto está envolvido. Ler essas mensagens é parte do trabalho. Conferir o resultado com `SHOW TABLES`, `DESCRIBE` e `SHOW CREATE TABLE`, e corrigir com `ALTER TABLE`, completa o ciclo.

Ao final desta aula, o modelo que estava no papel na Aula 05 existe no MySQL, com as regras da clínica protegidas pelo próprio SGBD.

18. Atividade da semana

1. Percorrer o roteiro `laboratorio-bd1-aula06.md` até criar as seis tabelas da clínica sem erro.
2. Escrever o script DDL da biblioteca comunitária (seção 15) e guardá-lo em um arquivo `biblioteca.sql`.
3. Responder ao questionário da Aula 06 no Google Classroom — 10 questões objetivas, com prazo até a data da A1.

Guarde os dois scripts: o da clínica e o da biblioteca. O arquivo `init.sql` exigido no trabalho da A2 tem exatamente esse formato.

19. Preparação para a Aula 07

Na próxima aula o banco criado hoje recebe dados: `INSERT` para inserir, `UPDATE` para corrigir e o primeiro `SELECT` para consultar uma tabela. Chegue com:

- o banco `clinica_veterinaria` criado e as seis tabelas funcionando;
- o contêiner `mysql-bd1` subindo sem erro;
- a dúvida anotada de qualquer passo que não tenha funcionado.

Uma pergunta para levar na cabeça: se `animal.tutor_id` é obrigatório e precisa existir em `tutor`, **qual linha precisa ser inserida primeiro?**

Referências

CHAMBERLIN, Donald D.; BOYCE, Raymond F. SEQUEL: a structured English query language. In: ACM SIGFIDET WORKSHOP ON DATA DESCRIPTION, ACCESS AND CONTROL, 1974, Ann Arbor. **Proceedings** [...]. New York: ACM, 1974. p. 249-264.

CHEN, Peter Pin-Shan. The entity-relationship model: toward a unified view of data. **ACM Transactions on Database Systems**, New York, v. 1, n. 1, p. 9-36, mar. 1976.

CODD, Edgar F. A relational model of data for large shared data banks. **Communications of the ACM**, New York, v. 13, n. 6, p. 377-387, jun. 1970.

DATE, Christopher J. **Introdução a sistemas de bancos de dados**. 8. ed. Rio de Janeiro: Elsevier, 2004.

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 6. ed. São Paulo: Pearson, 2010.

HEUSER, Carlos Alberto. **Projeto de banco de dados**. 6. ed. Porto Alegre: Bookman, 2009.

ORACLE CORPORATION. **MySQL 8.4 reference manual**. Redwood City: Oracle, 2026. Disponível em: <https://dev.mysql.com/doc/refman/8.4/en/>. Acesso em: 13 set. 2026.

SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. **Sistema de banco de dados**. 7. ed. Rio de Janeiro: LTC, 2020.